

## Pre-Merge Audit — Governing Prompt

---

### Pre-Merge Audit — Governing Prompt

1. Scope Statement
2. Goal
3. Inspection Surface
4. Function Count & Classification
5. Per-Function Reporting Requirements
6. Proposed Mergers to Analyse
7. Additional Findings Required
8. Required Report Organization
9. Final Report Format
10. Governing Rules

## Pre-Merge Audit — Governing Prompt

---

**Subject:** Complete pre-merge audit of the Vercel serverless functions in the EXZIBO project **Mode:** Analysis only — no code changes

---

### 1. Scope Statement

---

Perform a complete pre-merge audit of the Vercel serverless functions in this project. This task is analysis only. Do not modify, delete, rename, move, merge, refactor, install, deploy, or generate production code.

## 2. Goal

---

Determine whether the current Vercel functions can be safely consolidated without breaking:

- public API paths
- frontend calls
- authentication
- authorization
- database operations
- image uploads
- Upstash locking
- realtime events
- Vercel deployment behaviour

## 3. Inspection Surface

---

- `api/*.js`
- `api/_lib/*`
- `vercel.json`
- `vite.config.js`
- `server.js`
- `src/lib/db.js`
- all direct `fetch()` calls outside `src/lib/db.js`
- `src/db/*`
- Drizzle schema and migrations
- Better Auth configuration
- Upstash rate-limit and lock logic
- Cloudflare realtime event dispatch
- Cloudflare R2 upload logic
- all server-side environment-variable references

## 4. Function Count & Classification

---

First, determine the exact physical Vercel function count.

Clearly distinguish:

- physical Vercel functions
- `vercel.json` rewrites
- shared utility files
- Vite development middleware
- Express routes
- Cloudflare Workers and Durable Objects

## 5. Per-Function Reporting Requirements

---

For every physical Vercel function, report:

1. Exact file path
2. Public API paths and rewrites
3. Supported methods and actions
4. Frontend callers with exact file paths
5. Request and response contracts
6. Database tables read or written
7. Shared helpers and external services used
8. Environment variables used
9. Security boundary — public / authenticated / restaurant-scoped / role-protected / superadmin-only
10. Upstash rate-limit, idempotency and lock keys
11. Realtime events sent
12. R2 operations performed
13. Runtime-specific dependencies
14. Side effects, cleanup tasks and destructive operations
15. Duplicate implementations in `vite.config.js` or `server.js`

## 6. Proposed Mergers to Analyse

---

- **A.** `settings.js` + `team.js`
- **B.** `menu.js` + `content.js`
- **C.** `orders.js` + `bookings.js`

For each merger, check:

- runtime compatibility
- HTTP method and action collisions
- `vercel.json` rewrite collisions or ordering risks
- request parsing compatibility
- response-format compatibility
- CORS compatibility
- authentication and permission-boundary compatibility
- database transaction separation
- Upstash key collision risk
- realtime event collision risk
- payload-size and memory differences
- imported dependency and bundle-size risk
- timeout risk
- whether all existing frontend callers can remain unchanged
- whether the merged handler can remain a thin router using separate service modules
- exact safeguards required

**Verdict scale (one per merger, with file-path evidence):**

- SAFE
- SAFE WITH CONDITIONS
- DO NOT MERGE

## 7. Additional Findings Required

---

- dead API calls
- missing production endpoints
- duplicate aliases
- route or action collisions
- public write endpoints without authorization
- unprotected superadmin operations
- Drizzle-managed tables missing from `schema.ts`
- development and production behaviour differences
- obsolete Supabase or migration logic

- any function that must remain isolated

## 8. Required Report Organization

---

Separate the report into:

- **A.** Verified current facts
- **B.** Risks found
- **C.** Merger evaluation
- **D.** Recommended architecture

## 9. Final Report Format

---

1. Executive conclusion
2. Exact physical Vercel function count
3. Current function inventory
4. Frontend dependency map
5. Security-boundary summary
6. Duplicate implementation summary
7. Merger A analysis
8. Merger B analysis
9. Merger C analysis
10. Recommended final physical function count
11. Old-to-new function mapping
12. Public route compatibility plan
13. Shared modules that should be extracted
14. Safe migration order
15. Test checklist
16. Rollback plan
17. Final go/no-go recommendation

## 10. Governing Rules

---

- Do not recommend merging only to reduce the function count.
- Do not merge Better Auth, OAuth callbacks, R2 uploads, Cloudflare realtime, or destructive database operations unless the code proves it is safe.

- Preserve current public API URLs and frontend callers where possible.
- Do not guess missing permissions.
- Report security gaps, but do not combine security remediation with the merge implementation.
- Do not recommend deleting or renaming any action until every caller has been searched.
- Do not make any code changes.
- Keep the report focused on evidence needed for the merge decision.